

生存変数解析 — 「この値はまだ使われるか」を解く

今日のゴール

「この地点から先で、このレジスタの値がまだ読まれるか」を、フローグラフ全体にわたって求める。後ろ向きのデータフロー解析を 1 つ作る。

1 この回の位置づけ

- ・ 最適化発展シリーズの第 5 回。必須の前提は O2(フローグラフ) と O4(レジスタ割り当て)
- ・ この回は命令数を減らさない。解析そのものが成果物である
- ・ O6(コピー伝播と死コード除去) がこの解析を使う

2 なぜ O4 の後なのか

O3 の `is_dead_after` は、ラベルや分岐に出会うと「ここで基本ブロックが終わる → 判断できない」と答えて畳み込みをあきらめていた。

前向きに 1 行ずつ見るだけなので、ラベルの先が分からないのである。フローグラフの上で解けば、ブロックを越えて答えられる。

ただし、O4 を通す前の出力に対しては、解いても何も出てこない。コマ 16 のコード生成は `a0/a1` の 2 本しか使わず、値をすぐスタックへ逃がす。レジスタの生存区間が数命令しかないので、ブロックの境界をまたいで生きているレジスタが存在しないのである。

O4 で変数を `s1~s11` に置いて初めて、「ループの間ずっと生きている値」がレジスタの上に現れる。

重要

解析が何を見つけられるかは、コードの表現の仕方によって決まる。同じ解析でも、表現が変われば結果が変わる。この回では、それを数字で確かめる。

3 生きている、とは

あるレジスタが**生きている** (live) とは、「その地点より先で、書き換えられる前に読まれる」ことをいう。

```
li a0, 5          # ここで a0 は死んでいる（この後すぐ上書きされるなら）
li a0, 7
add a1, a0, a0    # ここでは a0 は生きている（直後に読まれる）
```

生きていない値は、計算する必要がない。これが死コード除去の根拠になる。

4 命令ごとの def と use

まず、命令 1 つが「何を書き」「何を読む」かを決める。

命令	def(書く)	use(読む)	注意
add rd, rs1, {rd}		{rs1, rs2}	算術一般
li rd, N {rd}		{}	定数は use に ならない
lw rd, N(rs) {rd}		{rs}	ベースを読む
sw rs2, N(rs) {}		{rs2, rs1}	ストアはレジ スタに書か ない
beqz rs, L {}		{rs}	飛び先ラベル は外す
call f	caller-saved すべて	a0～a7	呼び出し規約 から決まる
ret	{}	a0, ra, callee-saved すべて	呼び出し元へ 返す義務が ある

4 つ、間違いやすいところがある。

ストアの **def** は空である。sw a0, -24(s0) が書き込む先はメモリであって、レジスタではない。ここを {a0} にしてしまうと、まだ使う値を「死んでいる」と判断して消してしまう。

call はレジスタを壊す。呼ばれた側が自由に使ってよいレジスタ (a0～a7, ra, t0～t6) を全部 def 扱いにする。「呼び出しの向こう側で何が起きるか分からない」ことを、そのまま表す。

call の **use** は安全側に倒す。その関数が引数を何個取るかは、call f という命令からは分からない。だから a0～a7 を全部 use にする。実際より多く見えるが、少なく見積もるより安全である。

ret は **callee-saved** を読む。呼び出し元は s0～s11 と sp が保たれていることを期待している。つまり関数の出口では、これらが生きている。

これを忘れると、エピローグの復帰が死コードとして消える。

```
.L1:
    ld s1, -40(s0)      # s1 は「この先で読まれない」ように見える
    ld s0, 32(sp)       # → 消される → 呼び出し元の s1 が壊れる
    ret
```

main だけのプログラムでは気づきにくい。strops のように関数を呼ぶプログラムで、呼び出し元のループ変数が壊れて無限ループになるという形で表面化する。

5 ブロック単位にまとめる

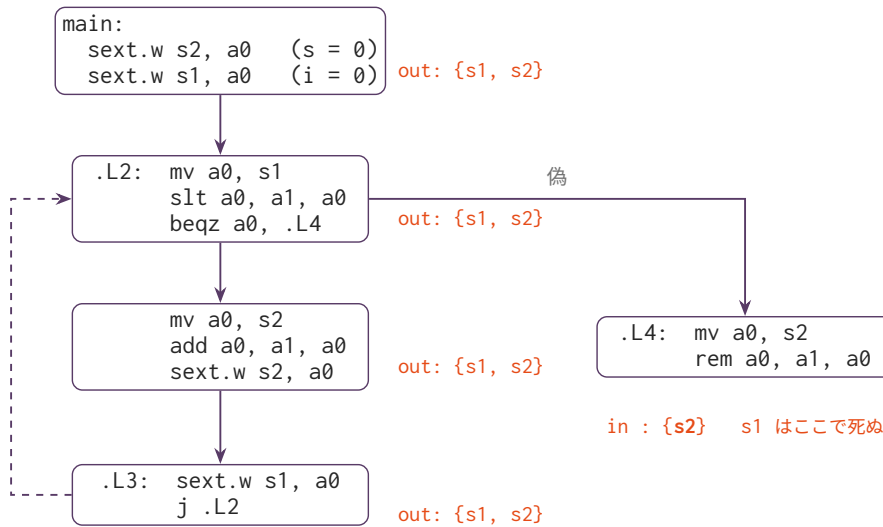
ブロックの中は一本道なので、先頭から順になめれば def と use が決まる。

```
use  ... ブロックの中で「書かれる前に」読まれるレジスタ
def  ... ブロックの中で書かれるレジスタ
```

「書かれる前に」が肝である。

```
mv  a0, s1      # s1 は use
li  a0, 5       # a0 に書いてから
add a1, a0, a0  # 読んでいるので、この a0 は use ではない
```

```
for (i=0; i<200; i=i+1) s = s + i;
```



方程式

$\text{live_out}[B] = \cup \text{live_in}[S]$ (S は B の後続)
 $\text{live_in}[B] = \text{use}[B] \cup (\text{live_out}[B] - \text{def}[B])$
 use は「書かれる前に読まれる」レジスタ、def は「書かれる」レジスタ

なぜ 1 回では終わらないか
 .L3 の live_out は .L2 の live_in から来る。ところが .L2 の値は、後方辺をたどって .L3 の結果に依存する。だから変化がなくなるまで繰り返す (不動点反復)

図 1 後ろ向きの反復で live_in / live_out を求める

6 方程式を解く

$\text{live_out}[B] = \cup \text{live_in}[S]$ (S は B の後続)
 $\text{live_in}[B] = \text{use}[B] \cup (\text{live_out}[B] - \text{def}[B])$

「後続で生きているものは、自分の出口でも生きている」「入口で生きているのは、自分が読むものと、自分が書き換えずに通す (後で使われる) もの」と読める。

後ろ向きなのは、**live_out** が後続から決まるからである。情報は制御の流れと逆向きに伝わる。

7 なぜ 1 回では終わらないか

ループがあると、1 周では解けない。

.L3 の live_out は .L2 の live_in から来る
 .L2 の live_in は … 後方辺をたどって .L3 の結果に依存している

お互いに依存しているので、変化がなくなるまで繰り返す。これを不動点反復という。集合は増える一方なので、必ず止まる。

8 実装

Step	実装対象	内容
1	<code>def_use</code>	上の表
2	<code>block_def_use / solve</code>	方程式を不動点まで解く
3	<code>live_after</code>	命令単位に展開 (O6 が使う)
4	<code>live_across_calls</code>	呼び出しをまたいで生きるレジスタ

呼び出し規約の定数と可視化 (`annotate`) は完成済みである。

```
python3 golden.py          # 割り当ての前後で解析結果を比べ、図を書き出す
```

9 テスト

```
python3 check.py          # def_use と方程式の解の単体テスト（未実装は SKIP）
python3 golden.py         # bench/*.c で「割り当て前は空振り、割り当て後に効く」を確認
```

`check.py` の Step 2 にはループを含む例が入っている。1 回なめただけでは解けないことを、ここで確かめる。

10 測ってみると

ブロックの境界で生きているレジスタの平均本数 (`sp` / `s0` / `ra` は常に生きているので数えない)。

ベンチマーク	ブロック	割当なし	割当あり	値を運ぶ s レジスタ
bubble	14	0.07	2.14	s1, s2, s4
fib_rec	6	1.50	1.50	(なし)
loop_sum	6	0.17	1.50	s1, s2
matmul	20	0.05	1.95	s1, s2, s3, s4
strops	18	1.56	3.67	s1, s2, s3, s4
平均		0.67	2.15	

`matmul` は割り当て前 **0.05**。20 ブロックのうち、境界で値を運んでいるレジスタはほぼ皆無である。解析器としては正しく動いているのに、**得られる情報が無い**。

割り当て後は 1.95 になり、`s1`～`s4` が 3 重ループの添字と合計を運んでいることが見える。

`fib_rec` だけ変わらないのは、**O4** が「割に合わない」と判断して 1 つも昇格しなかったからである (**O4** コマ 2)。

11 呼び出しをまたいで生きるもの

bubble	呼び出しなし
fib_rec	call 3回 callee-saved が要るもの: (なし)
loop_sum	呼び出しなし
matmul	呼び出しなし
strops	call 2回 callee-saved が要るもの: s1, s3

`strops` の `s1` と `s3` は、`my_strlen(s)` や `count_char(s, 111)` を呼んだ後もまだ使われる。だから callee-saved に置く必要がある — 解析がそれを裏づけている。

`fib_rec` が「(なし)」なのは、**O4** が昇格しなかったので保つべき値が無いからである。**O4** の判断が正しかったことを、**O5** の解析が確認している。

12 発展課題

1. **call の use を正確にする:** いまは `a0` ~ `a7` を全部 use にしている。関数の引数個数が分かれば減らせる。何を見れば分かるか。減らすと結果はどう変わるか
2. **干渉グラフ:** 同じ地点で同時に生きている 2 つのレジスタは、同じレジスタに割り当てられない。この関係をグラフにする。O4 で変数が 11 個を超えたときに必要になる
3. **到達定義解析:** 生存解析と逆向き (前向き) の解析。「この地点の値はどの命令が書いたか」を求める。方程式はどう変わるか
4. **O4 へ戻す:** `live_across_calls` を使えば、O4 の退避を「本当に必要なレジスタだけ」に減らせる。`strops` で何命令減るか
5. **解の一意性:** 空集合から始めて増やしていく代わりに、全集合から始めて減らしていくとどうなるか。同じ答えになるか

補足

コラム: データフロー解析は「同じ枠組み」の family である。 生存変数解析・到達定義解析・利用可能式解析・定数伝播は、どれも「ブロックごとの局所的な情報を、方程式で全体に伝える」という同じ形をしている。違うのは、向き (前向き/後ろ向き) と、合流点での演算 (和集合をとるか、積集合をとるか) だけである。

解析	向き	合流
生存変数	後ろ向き	U
到達定義	前向き	U
利用可能式	前向き	∩

1 つ書ければ、残りは枠組みの当てはめで書ける。この回で作ったのは、その最初の 1 つである。